答辩知识手册

把项目讲透
把问题答穿

册一 · 项目知识要点　|　册二 · 答辩实战问答（58 题）

智维 Agent（SmartOpsAgent）　可信 AI 智能运维与知识协同平台

核心主张

AI 负责思考，人负责授权，系统负责留痕。

赛事	**2026 第八届全球校园人工智能算法精英大赛 • 河南赛区 • 算法创新赛**
本册性质	**可直接背诵的知识要点 + 逐题口播答案**
数字口径	**全部来自代码实测，可当场自证（v1.0 • 2026-11-02）**
上场前	**只看三样：一页数字卡 • 真相清单 • 三个为什么**

打印建议：A4 双面（长边翻转）· 缩放 100% · 黑白即可，正文不使用深色底

怎么用这本册子

册一回答"这个项目是什么"——读完你能从业务背景讲到某一行为什么这么写。
册二回答"老师会问什么、我该怎么答"——每题给"老师想听什么 + 口播答案 + 追问链 + 千万别这么说"。
最重要的一节是册一第 8 节「真相清单」：8 个"没做完"的地方，**主动说出来是加分，被戳穿是事故。**

目录

节	内容	页
封面	封面 + 打印顺序	1
目录	怎么用这本册子	2
册一	项目知识要点手册	3 起
— 0	30 秒讲清这个项目	
— 1	它到底解决什么问题（三个真痛点）	
— 2	系统全景：6 个服务与真实规模	
— 3	必须能讲的功能清单	
— 4	核心算法知识点（命根子）	
— 5/6	后端要点 / 前端要点	
— 7	数据、测试、部署的真实数字	
— 8	真相清单：哪些是真做、哪些是基线	
— 9/10	一页数字卡 / 三个必须默写的为什么	
册二	答辩实战问答手册（58 题）	
— 开场	3 分钟陈述稿	
— 一	定位与价值 5 题	
— 二	算法深挖 12 题	
— 三	架构与工程 8 题	

节	内容	页
一四	真实性与边界 12 题（最危险）	
一五	安全与伦理 5 题	
一六	产品与商业 6 题	
一七	团队与个人 4 题	
一八	刁钻题 6 题	

三条铁律

一、不报任何准确率 / F1 数字——实验全是 NOT RUN，硬报就是造假。

二、被问到没做到的，用「承认 → 给已知 → 划边界 → 给路径」四步接住。

三、所有数字以册一第 9 节「一页数字卡」为准，四份材料不许打架。

册一

项目知识要点·把项目彻底讲透

项目知识要点手册.md

智维 Agent·项目知识要点手册

用途：答辩前把项目彻底讲透。**所有数字均来自代码实测，可当场自证。**

版本 v1.0 · 2026-11-02 | 代码锚点格式 `文件路径:行号`

0. 30 秒讲清这个项目（电梯版，背到脱口而出）

我们做的是**智维 Agent**——一个敢让它动手的运维智能体。

现在企业用 AI 运维有个死结：**通用大模型会"说"，但不敢让它"做"**——因为它可能引错版本的文档、可能声称"已经重启成功"其实没成功、可能一条命令把生产环境干掉。

我们的解法是**三道锁一本账**：**证据锁**（每句话能追溯到知识条目）、**风险锁**（高危操作必须人工授权）、**边界锁**（命令白名单 + 黑名单 + 沙箱），最后**一本审计账**全程留痕。

技术上，检索用版本感知的混合检索，结果核查用**四级纠察**并能识别"假成功"，可信度用 trust_score 聚合。

全栈自研：后端 SpringBoot 80 张表 296 个接口，前端 Vue3 双入口 26+ 页面，AI 侧 Python 混合检索与纠察内核。

一句话：**AI 负责思考，人负责授权，系统负责留痕。**

1. 它到底解决什么问题

1.1 场景

中小企业的 Linux 服务器运维：没有专职 SRE，一个人管几十台机器，出问题靠"百度 + 试命令 + 祈祷"。

1.2 三个真痛点（评委最吃这套，因为它对应三个算法设计）

痛点	具体表现	我们的对应设计
知识错配	搜到 Ubuntu 22.04 的答案，但机器是 CentOS 6，`systemctl` 根本不存在	**版本感知检索**（版本过滤 + 版本一致性核查）
假成功	AI 说"已重启 nginx"，其实服务根本没起来	**四级纠察 + 假成功状态核验**
不敢授权	一条 `rm -rf` 或 `iptables` 就能毁掉生产环境	**风险四档分级 + 双闸 + 命令白/黑名单**

1.3 用户与角色

`ROLE_ADMIN` / `ROLE_OPERATOR` / `ROLE_PASSWORD_CHANGE`（`backend/.../AuthController.java:134`）；团队空间内另有 `owner` 起的多级授权（`TeamWorkspaceService.java:47`），全项目 **139 处** `teamId` 引用，是真多租户。

---

2. 系统全景（背下这张表，架构题不会失分）

服务	技术	端口	职责	真实规模
backend	SpringBoot 3.3.5 + Java 17	8080	业务主干：权限、服务器、运维会话、哨兵、审计、模型网关、决策闭环	219 个 Java 文件 / 17,981 行 / 45 Controller / 296 接口 / 80 张表
ai-service	FastAPI（Python）	8001	AI 对话、RAG 编排、SSE 流式	3 个端点（health / chat / chat-stream）
knowledge-service	Python	8002	知识入库、切片、嵌入、检索、证据	13 个单测全通过
agent-service	Python	8003	Agent 内核（planner/executor/inspector/policies）	库形态 + 135 用例
frontend	Vite 7 + Vue3 + Element Plus	5173 / 5174（双入口）	用户层 + 管理员层	146 个 .vue / 27,983 行
基础设施	MySQL 8.4 + Redis 7.4	3306 / 6379	存储与缓存	docker-compose 只有这两个服务

一句话记住：`浏览器 → Nginx → backend(8080) / ai-service(8001) → knowledge(8002)`；Python 侧**全部零密钥可跑**（`LLM_PROVIDER=mock`）。

3. 你必须能讲的功能清单（真存在的，不许讲没有的）

能力	在哪	一句话说明
AI 对话（流式）	`frontend/src/api/stream.ts`	手写 SSE：fetch + ReadableStream + TextDecoder，按 `\n\n` 切帧，解析 `thinking/token/done`
思考过程可视化	`AiChatPanel.vue:8-16`	6 个阶段逐步点亮，最后一步脉冲，结束全变 done
知识库（系统）	`KnowledgeView.vue`	本地静态条目，页面角标明确写"未人工核验"
个人知识库	`PersonalKnowledgeView.vue`	**真后端 CRUD**：建库/导入/重建索引/导出/归档，条目带 version 与 verificationStatus
运维执行（预览→审批→执行）	`api/operations.ts`	`previewMultiExec → executeMultiExec`，**审批后才真正发送命令**
运行记录	`RunHistoryView.vue`（778 行）	按 Trace ID/工具/状态/时间查，展示命令、退出码、stdout/stderr
SSH/SFTP/隧道终端	`backend/operations/` + `sshd-core 2.12.1`	**真实现**，WebSocket `/ws/operations/terminal`
哨兵告警与自愈脚本	`backend/sentinel/`（15 个类）	任务/采样/告警/规则/脚本版本/运行/调度 + guardian 证据 + 学习画像
AI 决策闭环	`DecisionLoopService`	五阶段 **ANALYZE → MEMORIZE → DECIDE → REVIEW → LEARNED**，复盘回写 `operator_memory` 置信度
模型网关	`ModelGatewayService`	对外 `/v1/chat/completions`，API Key 鉴权 + 配额 + 用量统计
Agent 指挥中心	`/agent`、`/agent/trace`	Agent 编排与执行轨迹，`requiresHuman` 人工审批勾选项
安全治理	`/admin/*` + 管理员层	用户/组织/角色/菜单/字典/通知/审计/**TOTP MFA**
视觉化	`ParticleWhale.vue`、`milky-way-scene.js`	Three.js 粒子鲸 + **GPGPU 离屏迭代**星海（ShaderMaterial 模拟场）
数据图表	ECharts 4 个组件	折线 / 堆叠柱 / 环形饼 / 指标卡，实际用于模型分析页
语音输入	`domain/chat/speechInput.ts`	Web Speech API，`lang:'zh-CN'`，**只填文本不上传音频**
截屏取证	`domain/chat/visualCapture.ts` + `chatAttachments.ts`	`getDisplayMedia/getUserMedia` → canvas → PNG → **真上传**后端，返回 ocrText/confidence/privacyStatus

4. 核心算法知识点（算法创新赛的命根子，必须滚瓜烂熟）

4.1 知识入库流水线

```
文件/文本 → 解析(.txt/.md/.pdf) → 切片 → 嵌入 → 内存存储
```

- 切片：`chunk_size=800, overlap=120`（`knowledge/config.py:6-7`），字符滑窗 `start = end - overlap`
- 嵌入：**HashEmbedding，256 维**（`embedding/hash_embedding.py`）—— 对每个 token 取 `sha256(token)[:4] % 256` 累加计数后 L2 归一化
- 存储：`InMemoryVectorStore.items` 列表，**进程内存，重启即失**

4.2 检索三件套

① BM25（`retrieval/bm25.py`）

- `k1=1.5`、`b=0.75`；`idf = log(1 + (N-df+0.5)/(df+0.5))`
- 分词：`re.findall(r"[\u4e00-\u9fff]+|[a-zA-Z0-9_]+")` → **中文连续汉字当成一个 token，不是真分词**（无 jieba 依赖）
- df **每次查询全量重算**，无倒排索引

② 向量路（`retrieval/vector_store.py:59-73`）

- 逐条 cosine 线性扫描；**无 FAISS / HNSW / Milvus**

③ 融合（`retrieval/hybrid.py:89-101`）

```
score = 0.6 × vector_norm + 0.4 × bm25_norm
```

- 归一化 = **各自除以本路最高分**（max 归一，**不是 min-max，也不是 RRF**）
- 按 `chunk_id` 合并去重，单路缺失记 0，`top_k=10`

4.3 版本感知（我们最独特的点）

- version 是 chunk 上的字符串字段，默认 `"latest"`（`config.py:17`）
- 过滤逻辑（`version_filter.py:8-15`）：空或 `latest` 不过滤，否则**只保留 version 完全相等**的
- **举例（评委最爱听例子）**：问"Ubuntu 22.04 如何重启 nginx？"，带 `version=ubuntu-22.04`，CentOS 6 的 `service nginx restart` 就不会混进答案
- 兜底：证据版本与环境版本不一致 → inspector 打 `VERSION/WARNING`

4.4 Reranker（默认启用）

`reranker/lexical_reranker.py:43-46`：`final_score = 0.7 × result.score + 0.3 × 词元重合率`，取 `top_k=5`

4.5 trust_score（可信度聚合，必背公式）

```
score = 0.5 × evidence_score      # 各证据 score 的均值
      + 0.3 × support_score       # 是否被支持（1.0 / 0.0）
      + 0.2 × coverage            # min(证据数/3, 1)
```

（`knowledge/evidence/trust_score.py:28-37`，裁剪到 [0,1]，保留 4 位小数，无证据返回 0.0）

> **三个分量为什么这么定**：证据质量是基础（0.5），是否被支持是硬门槛（0.3），覆盖度防止"一条证据撑起一个结论"（0.2）。

4.6 四级纠察（Inspector，核心创新）

`agent/inspector/pipeline.py`

级	做什么	关键阈值
L1 **Rule**	声明风险 < 期望风险 → HIGH；HIGH/CRITICAL 且未获批 → CRITICAL；同工具调用 > 10 次 → HIGH（循环检测）；无证据却调用状态变更类动作 → HIGH	`loop_threshold=10`
L2 **Evidence**	词元重合率 ≥ 0.5 → SUPPORTED；≥ 0.2 → PARTIALLY_SUPPORTED；否则 UNSUPPORTED；证据数 < 1 → WARNING；版本不符 → VERSION/WARNING	4 级标签：SUPPORTED / PARTIALLY / UNSUPPORTED / CONTRADICTED
L3 **AI Judge**	**可插拔钩子，默认 passthrough 不调模型**	`judge_fn is None → return []`
L4 **Verifier**	命中"已解决/已恢复/fixed/ok"等 9 个模式，但 `actual_state["resolved"]==False` → **假成功**	`false_success=True`

聚合结果：无问题 → PASS；有问题 → WARN；严重度 HIGH/CRITICAL → VIOLATION；**假成功 → REJECTED**，并建议"触发 Re-plan"。

4.7 风险分级与安全双闸

- 四档：`LOW / MEDIUM / HIGH / CRITICAL`（`policies/risk.py:4-12`）
- 映射：`CRITICAL = {server.delete, database.drop}`；`HIGH = {server.restart, server.stop, server.delete, shell.execute}`；`server.*` → MEDIUM
- **双闸**：CRITICAL 一律禁止；HIGH 且未标记 `requires_approval` → 直接拒绝（`policy_engine.py:27,36-46`）
- **命令白名单**（`tools/builtin/shell.py:106-132`）：Linux 19 个（cat df du free grep head hostname id ip ls netstat pgrep ps pwd ss tail uname uptime whoami）+ Windows 3 个
- **黑名单**：`rm rmdir del format shutdown reboot kill pkill killall iptables nft systemctl service mount umount chmod chown useradd userdel passwd dd mkfs docker kubectl terraform`

- 还禁元字符 `| && || ; > >> < \` $(${ 与危险参数 --force --delete --privileged --device --cap-add --pid=host --network=host`

- 执行护栏：`timeout=20s`、输出截断 20,000 字符、强制 `shell=False`（不给 shell 解释）

5. 后端知识要点（被问"是不是只有个壳子"时的弹药）

要点	事实		
认证	**不是 JWT**——HttpSession + JSESSIONID（30 分钟）+ Spring Security；`SecurityConstants.TOKEN_PREFIX="Bearer "` 定义了但无签发逻辑		
MFA	**TOTP 真实现**（`TotpService` + `account_mfa` / `account_mfa_recovery`），人机验证支持本地图形点选与 Cloudflare Turnstile 双 Provider		
审计	两张表：`security_audit_event`（actor/action/target/outcome）、`operation_audit_event`（含 approval_state）；`SecurityAuditService` 硬编码 **62 种受控动作**，结果 SUCCESS/FAILURE/DENIED		
ORM	MyBatis（6 Mapper / 4 XML）与 **JdbcTemplate（45 个文件）混用**		
Redis	依赖与配置齐全，但 **RedisTemplate 引用数 = 0**（骨架）		
测试	**89 个测试文件 / 8,761 行 / 428 个 @Test**，含 13 个端到端集成测试与 3 个安全专项测试		
质量	全量 grep `TODO	FIXME	HACK` 命中 **0 处**		

6. 前端知识要点（被问"界面是谁做的"时的弹药）

- 双入口：用户层 5173 / 管理员层 5174，共用 `vite.shared.mjs`（代理 `/api` `/ws`，`/ai` 单独转发 8001）
- 手动分包：three / zrender / echarts / xterm / vue / axios 拆开
- 状态管理：Pinia 7 个 store，`aiChat.js` 就有 17.6 KB
- 错误处理：`request.ts`（498 行）统一拦截，覆盖 400/401/403/404/409/**428 人机验证**/429/422/500/502-504；401 派发 `smartops:unauthorized` 事件由路由清会话跳登录
- SSE 刻意不走 axios（axios 读不了流），用原生 fetch + ReadableStream，支持 AbortSignal
- 设计系统：仓库根 `design-system/MASTER.md`（87 行）定义颜色/排版/栅格/动效/无障碍；深色为主 + 浅色令牌切换

- **诚实点**：`tsconfig.json` 是 `strict:false`，且 include 只覆盖 api/domain/utils/types，**views 与 components 不在类型检查范围**

7. 数据、测试、部署（全部可当场复现）

项	真实数字
Python 测试	**135 用例 = 111 通过 / 24 跳过 / 0 失败**（54.43s）
├ knowledge	13 个**全通过**（纯内存，无外部依赖）
├ training	36 个（数据集 schema/校验/泄漏/门禁）
├ inspector	20 个（规则与证据检查器、事件总线、持久化）
├ agent	18 个（skill 加载、shell 安全）
└ e2e / integration / api	5 / 6 / 15（**跳过是因后端未启动或缺依赖，不是 mock 掉**）
另	ai-service 约 22 个（对话 / RAG / LLM 集成，计入总 135）
后端测试	428 个 @Test + 4 个参数化
数据集	7 个 JSONL / **共 14 条** / `status` 全部 `DRAFT`
实验文档	17 个 md，**9 个显式 NOT RUN，无任何真实评测指标**
部署	compose 只有 MySQL + Redis；K8s 有 6 个 deployment + ingress + hpa，**无 Helm**

唯一真实的评测数字：`111 passed / 24 skipped`、`CUDA: unavailable`、`GPU count: 0`——文档自己注明"**工程回归结果，不是模型实验结果**"。

8. 真相清单：哪些是真做、哪些是基线（这一节决定你会不会翻车）

主动说出来 = 真懂；被戳穿 = 造假。

模块	真实做法	生产该怎么做	一句话答法
Embedding	SHA256 哈希桶伪嵌入 256 维；`lower().split()` 对中文退化为单 token	bge-m3 / text-embedding-3 + 中文分词	"零依赖可复现的哈希基线，接口可替换；中文语义目前由词法路与重排兜底"
BM25	正则伪分词，每查全量重算 df	jieba/IK + 倒排索引	同上，且**这是我们有 reranker 的原因**
融合	0.6/0.4 max 归一化加权和	RRF 或 learned-to-rank	"小语料下 max 归一比 min-max 稳，参数留了网格搜索接口"
向量检索	内存线性扫描	FAISS/HNSW + 持久化	"千级 chunk 内线性扫描够用，规模化换 FAISS"
Planner	**3 条 if/elif 关键词规则，最多 1 步**	LLM Planner（模板已写好）	"确定性基线先跑通可测；`PLANNER_PROMPT` 已就绪，替换成本是一个适配器"
AI Judge (L3)	**默认 passthrough，不调模型**	接 LLM 做语义判定	"**真正的安全靠 L1+L2+L4，不依赖 LLM**——这是设计选择，不是缺陷"
LLM	`provider` 默认 **mock**；设 `ollama` 且服务可达时 **真实调用 Ollama（qwen3:1.7b）**（Java 侧 `MiniAssistantEngine` 即默认连本地 Ollama）	真实 provider + 密钥托管	"零密钥可复现是刻意的：评委/老师没 key 也能跑通全链路；有 Ollama/Key 时切真模型只需一个环境变量"
端到端编排	**Python 侧 planner/executor 无调用方**；真正在跑的产品闭环是后端 `DecisionLoopService`（ANALYZE→MEMORIZE→DECIDE→REVIEW→LEARNED 五阶段）	统一编排	"Agent 内核（planner/executor/inspector）是可插拔算法框架，经 113 单测验证；产品级闭环走后端决策循环。两者**共享'证据锁/假成功检测'的设计理念，并非同一份代码**"
审批流	`ApprovalManager` 只有数据结构，**无编排调用方**	接入执行前拦截	"审批在产品侧由 preview→approve→execute 三段式实现"
沙箱	已实现（process/filesystem/network），**未被 executor 调用**	接入 executor	"沙箱模块已完成，接线是下一步"
Executor	**无重试 / 幂等 / 回滚**	补偿事务	"失败上报 inspector 触发 Re-plan；幂等回滚在路线图"
数据集	14 条 DRAFT，`schemaVersion` 类型不一致（4 个数字 / 10 个字符串）	千级样本 + 人工审核	"14 条种子集只用于证明链路打通，不报准确率"
记忆	内存 deque(50) / dict	持久化 + 检索	"轻量实现，够演示"

模块	真实做法	生产该怎么做	一句话答法
OCR	`NoopOCR.extract_text` 直接 `raise NotImplementedError`	PaddleOCR	"OCR 未实现，视觉走的是另一条 vision-event 链路"

9. 一页数字卡（打印下来，所有数字以这一页为准）

```
服务端口   backend 8080 | ai-service 8001 | knowledge 8002 | agent 8003 | 前端 5173/5174
后端       219 Java 文件 / 17,981 行 / 45 Controller / 55 Service / 296 接口 / 80 张表
           SpringBoot 3.3.5 + Java 17 + MySQL 8.4 + MyBatis+JdbcTemplate
前端       146 .vue / 27,983 行 / Vite 7 + Vue3 + Element Plus + Pinia(7)
Python     16,536 行 / 244 文件 / FastAPI + 自研 Agent 内核
测试       Python 135 (111 通过 / 24 跳过) | 后端 428 @Test | 共 563
算法常量   切片 800/120 | 嵌入 256 维 | BM25 k1=1.5 b=0.75 | 融合 0.6/0.4
           Reranker 0.7/0.3 (top5) | trust 0.5/0.3/0.2 | 纠察 4 级 | 风险 4 档
安全       白名单 22 条命令 | 黑名单 23 条 | 元字符禁用 | timeout 20s | 输出截断 20k
           审批双闸 | 审计 62 种动作 | TOTP MFA | Session 30 分钟
数据集     7 类 / 14 条 / 全部 DRAFT
实验       17 个文档 / 9 个 NOT RUN / 无真实指标
```

10. 三个必须能默写的"为什么"

1. **为什么做版本感知？** —— 运维知识有强版本耦合，Ubuntu 的答案在 CentOS 上是错的，通用 RAG 不管这个。**这是我们和"接个大模型就完事"的根本差别。**
2. **为什么做假成功检测？** —— Agent 最大的风险不是不会做，是**声称做成了其实没做成。**L4 状态核验专门抓这个：命中"已解决"话术但实际 `resolved=false` → REJECTED。
3. **为什么安全机制不依赖 LLM？** —— L1 规则、L2 证据、L4 状态核验都是确定性判定，**AI Judge 默认不调模型。**用不确定的模型去守确定的安全底线，是设计错误。

册二

答辩实战问答 · 58 题怎么答

答辩实战问答手册.md

智维 Agent · 答辩实战问答手册

每道题四段式：**老师想听什么 → 你照着说（口播稿）→ 他接着会追什么 → 千万别这么说**
口播稿按"能直接念出来"写，不是书面语。

开场陈述（3 分钟，背熟）

各位老师好，我们是【队名】，项目是**智维 Agent**。

我想先说一个现象：现在企业里 AI 已经被用来问问题了，但**几乎没人敢让它真正动手**。原因是三个不信任——**它用的知识可能是错版本的**（Ubuntu 的答案套到 CentOS 上），**它可能说"已经重启好了"其实没好**，**一条命令就可能把生产环境搞挂**。

我们的项目就是冲这三个"不信任"去的。解法是**三道锁一本账**：
证据锁——每句话能追到具体知识条目和版本号；
风险锁——高危操作必须人工授权，命令有白名单和黑名单；
边界锁——沙箱、超时、输出截断、元字符禁用。
最后**一本审计账**全程留痕。

技术上，检索是**版本感知的混合检索**，结果核查是**四级纠察**，其中最关键的是能识别**假成功**；可信度用 **trust_score** 三分量聚合。

这不是 PPT：后端 SpringBoot **80 张表 296 个接口**，前端 Vue3 **26 个以上页面**，AI 侧自研 Python 内核，**563 个测试用例**，而且**零 API Key 就能跑通全链路**。

最后一句总结我们的主张：**AI 负责思考，人负责授权，系统负责留痕。**

一、定位与价值（为什么做这个）

Q1 你们和普通"接个大模型做运维问答"有什么区别?

- **想听**：你是不是只是套壳。
- **说**：差别在**约束**，不在模型。通用问答只解决"会不会说"，我们解决"敢不敢做"。我们加了三层通用方案没有的东西：**版本感知**（防止知识错配）、**四级纠察 + 假成功检测**（防止谎报成功）、**风险分级双闸**（防止越权执行）。模型是同一个模型，**差别在模型外面那一层约束工程**。
- **追问** → Q2、Q14
- **别这么说**："我们模型更聪明"——你没训模型，一问就穿。

Q2 为什么选运维这个方向?

- **想听**：需求是真的还是编的。
- **说**：两个原因。一是**痛点强度高**：运维是少数"AI 出错就有真实损失"的场景，所以可信机制不是锦上添花而是刚需，这个方向最能验证我们的方法。二是**可验证**：运维操作有明确对错（服务起来没起来、端口通不通），不像写诗，这让"假成功检测"这类设计能被客观检验。
- **追问** → 你怎么验证的？→ 接 Q31
- **别这么说**："运维市场很大"——空洞。

Q3 最核心的创新点，只准说一个，你说哪个?

- **想听**：你有没有主次。
- **说**：**假成功检测**。Agent 最大的风险不是不会做，是**声称做成了其实没做成**——这比不做更危险，因为人会基于"已完成"去做下一步。我们的 L4 状态核验会比对"话术"和"实际状态"：命中了"已解决/已恢复/fixed"这类表述，但实际 `resolved=false`，就直接判 REJECTED 并触发重新规划。这个机制我查过，通用 Agent 框架里是缺的。
- **追问** → 怎么判定实际状态？→ Q17
- **别这么说**："我们创新点有三个……"（然后三个都讲不深）

Q4 你的目标用户到底是谁?

- **想听**：有没有真实用户画像。
- **说**：**没有专职 SRE 的中小企业**——一个人管几十台 Linux 机器，出问题靠搜索加试命令。他们有三个特征：付得起小钱、买不起整套 AIOps、最怕误操作。我们在系统里也对应做了设计：团队空间支持协作、审批流降低误操作风险、SSH 终端真实现可以直接上手。
- **追问** → 那大厂呢？→ "大厂有自研 AIOps 和专职团队，我们的差异化恰恰在**轻量可自部署**"
- **别这么说**："所有人都需要"。

Q5 做这个最难的地方在哪?

- **想听**：你真做过还是拼凑的。
- **说**：最难的不是让 AI 说话，是**让 AI 的每一句话都可追责、每一个动作都可回滚**。具体来说，**把"可信"做成可执行的工程约束**：怎么定义证据充分、怎么定义风险等级、怎么在没接大模型的情况下也能判定安全。我们最后的答案是——**安全判定不依赖 LLM**，规则和证据核验是确定性的，AI 只作为可选增强。
- **追问** → 不依赖 LLM 怎么保证覆盖率？→ Q21

二、算法深挖（这块是算法创新赛的主战场）

Q6 混合检索为什么是 0.6 / 0.4，不是 0.5 / 0.5?

- **想听**：参数是你调的还是随手写的。
- **说**：这组合是**设计取向**而不是网格搜索出来的：向量路承担语义泛化（用户问法和文档说法不一样时靠它），词法路承担**精确术语命中**（`nginx`、`iptables`、`ss` 这类专有名词，向量路容易糊）。所以给语义路更高权重 0.6，词法路 0.4 兜底。同时我们的**词法路本身做了强化**——加了词法重排 `0.7×原分 + 0.3×重合率`，等于把词法信号用两次。参数在配置里是可调项，网格搜索在我们的评测计划里。
- **追问** → 有对比实验吗？→ **诚实答**：没有（见 Q31）
- **别这么说**："我们实验得出 0.6 最优"——没实验就是没实验。

Q7 归一化为什么用 max 归一，不用 min-max 或 RRF?

- **想听**：你懂不懂归一化。
- **说**：min-max 对小样本**极不稳定**——两路分数的绝对量纲不同（BM25 可以到十几，余弦在 0~1），min-max 会让最低分那条直接变 0、最高分变 1，少量样本时容易被单个离群值拉偏。max 归一（各路除以本路最高分）保证**两路都落在 0~1 且最高分恒为 1**，量纲可比又不受离群值放大。RRF 确实是更鲁棒的排名融合方案，我们把它列为下一步——**但 RRF 丢弃了分数信息，而我们的 trust_score 要用到分数**，所以当前设计保留了分数。
- **追问** → 那 trust_score 怎么用这个分数？→ Q10
- **别这么说**："大家都这么用"。

Q8 版本感知具体怎么做？举个工作例子。

- **想听**：独特卖点是不是真的。
- **说**：每个知识切片带一个 `version` 字段，默认 `latest`。检索时如果指定了版本，就**只保留版本完全相等的切片**。举个例子：用户问"Ubuntu 22.04 怎么重启 nginx"，系统带上 `ubuntu-22.04`，那么 CentOS 6 的 `service nginx restart` 就不会进上下文——而这在通用 RAG 里是最常见的错误。还有第二道保险：如果证据版本和环境版本不一致，inspector 会打 `VERSION/WARNING`，即使漏过了检索也会被纠察抓出来。
- **追问** → 版本不匹配怎么办？→ "回退到 `latest` 并对答案标注版本不确认，绝不静默用错版本"
- **别这么说**："我们做了语义版本比较"——目前是字符串相等匹配，别吹。

Q9 BM25 的中文分词怎么处理的?

- **想听**：中文场景必问，最容易翻车。
- **说**：**坦白说，目前是规则伪分词**：用正则把连续汉字当一个 token、英文数字当一个 token，没有接 jieba。这是**零依赖可复现**的取舍——我们不希望评委或老师跑项目时还要装分词模型。代价是中文长 query 的召回会下降，我们的补偿手段是**两条**：一是向量路承担语义泛化，二是**词法重排会用 query 与内容的 token 重合率再打一次分**。生产环境换 jieba 或 IK 只需要替换 `tokenize` 一个函数。
- **追问** → 那中文效果怎么保证？→ 接上面"两条补偿"
- **别这么说**："我们做了中文分词优化"——没有，会被拆穿。

Q10 trust_score 三个权重 0.5 / 0.3 / 0.2 怎么定出来的?

- **想听**：公式是不是拍脑袋。
- **说**：三个分量对应三种失败模式。**0.5 证据质量**（各证据分的均值）——基础质量，权重最高；**0.3 是否被支持**——这是个硬门槛式的二值信号，防止"有证据但答非所问"；**0.2 覆盖度** `min(证据数/3, 1)`——防止"一条证据撑起一个结论"，三条以上就饱和。这个权重是**先验设计**，不是拟合出来的，我们也把它列为待标定项。
- **追问** → support_score 怎么算？→ **诚实答**：当前传的是"有没有检索到结果"，不是真的答案-证据蕴含判定，**这是明确的待改进项**（见 Q37）
- **别这么说**："通过大量实验标定"——没有。

Q11 四级纠察为什么需要四级，一级不够吗?

- **想听**：复杂度有没有必要。
- **说**：四级对应四类不同的失败，**互相不可替代**：L1 规则管**流程与权限**（该不该做、有没有授权、有没有死循环）；L2 证据管**依据**（结论有没有证据撑、版本对不对）；L3 AI 管**语义**（规则覆盖不到的语义矛盾）；L4 状态核验管**结果**（声称的完成是不是真的）。少了任何一级就有一类风险漏掉——**尤其 L4，几乎所有 Agent 框架都没有。**
- **追问** → 四级会不会太慢？→ "都是确定性判定，无模型调用，开销在毫秒级"
- **别这么说**："越多越安全"。

Q12 假成功检测具体怎么判定?

- **想听**：核心创新是不是真实现了。
- **说**：两层比对。第一层看**话术**：结果文本里有没有命中"已解决、已恢复、fixed、resolved、ok"这类 9 个完成态模式；第二层看**实际状态**：传入的 `actual_state["resolved"]` 是不是真的为 true。**话术说完成但实际没完成** → `false_success=True` → **直接 REJECTED**，并建议触发重新规划。这就是"声称做成了其实没做成"的机器判定。
- **追问** → 实际状态从哪来？→ **诚实答**：当前由调用方传入，关键词匹配而非真实探活；**真实探活（如查端口、查进程）是明确的下一步**
- **别这么说**："我们会真实探测服务状态"——目前不是。

Q13 Reranker 是什么方案?

- **说**：词法重排，`final_score = 0.7×检索分 + 0.3×词元重合率`，取 Top-5。它是**对混合检索结果的二次校正**，也是我们对中文伪分词的一种补偿。没有用 cross-encoder 或 BGE-reranker，因为那会引入模型依赖，破坏"零密钥可复现"。
- **追问** → 为什么 0.7/0.3？→ "以检索分为主、重合率做微调校正，重合率只是辅助信号不该喧宾夺主"

Q14 你的算法和主流 RAG 相比，创新的"增量"在哪?

- **说**：主流 RAG 的优化目标是"**找得更准**"，我们的增量是"**找得准之外，还要可追责、可判定、可拦截**"。具体三个增量：**版本感知**（运维领域特有，通用 RAG 不管版本）、**证据分级与可信度聚合**（把检索分变成可解释的可信度）、**执行后核验**（把 RAG 从"检索环节"延伸到"执行结果环节"）。**增量在环节延伸，不在单点精度。**
- **追问** → 那精度呢？→ 接 Q31 诚实答

Q15 切片 800 / 120 怎么定的?

- **说**：运维文档多是操作步骤，800 字符大约能容纳一个完整步骤加上下文，120 的重叠保证跨切片的命令不被切断。这是**针对运维文档形态的经验值**，不是网格搜索结果；切分策略本身是可配的。

Q16 为什么用哈希嵌入？效果能行吗?

- **想听**：是不是偷懒。
- **说**：**是刻意的取舍**，不是偷懒。哈希嵌入（`sha256` 分桶到 256 维）的价值是**零依赖、零密钥、可复现**——任何人克隆仓库就能跑通全链路，不需要下载模型、不需要 GPU（我们实测环境是 `CUDA: unavailable`、`GPU count: 0`）。代价是**它没有语义**，中文还会因为 `lower().split()` 退化成单 token。所以我们的语义能力实际由**向量路的近义匹配 + 词法重排 + BM25** 共同承担。生产环境换真 embedding 只需替换一个适配器类。
- **追问** → 那你承认语义检索现在基本没用？→ **诚实答**："在中文场景下语义贡献确实有限，所以我不会说我们的检索准确率有多高；我们强调的是**链路完整 + 机制可验证**。"
- **别这么说**："哈希嵌入效果很好"——必翻车。

Q17 四级纠察的输入输出是什么，能画一下吗?

- **说**：输入是四样：执行计划步骤、工具执行结果、检索到的证据列表、实际状态。输出是 `InspectorResult`：status（PASS/WARN/VIOLATION/REJECTED）、issues 列表（每条带 severity/category/reason/recommendation）、`requires_human` 布尔、以及 verification 记录。判定规则：无问题 PASS；有问题 WARN；严重度 HIGH/CRITICAL 转 VIOLATION；假成功直接 REJECTED。**任何 REJECTED/VIOLATION 或 HIGH/CRITICAL 都会置 `requires_human=true`。**

Q18 系统有几个服务，怎么通信?

- **说**：前端 Vite（用户层 5173、管理员层 5174）经 Nginx 代理到后端 SpringBoot（8080）和 AI 服务 FastAPI（8001）；AI 服务再通过 HTTP 调知识服务（8002）做检索。Agent 内核（8003）是库形态。基础设施是 MySQL 8.4 和 Redis 7.4。后端与 AI 侧之间还有内部 Token 鉴权。

Q19 后端为什么用 Java、AI 侧用 Python?

- **说**：**按优势分工**：Java/SpringBoot 承担事务、权限、审计、会话这类"强一致、强安全"的企业级能力，生态成熟；Python 承担检索、Agent、模型交互这类"算法密集、迭代快"的部分。两者通过 HTTP + 内部 Token 解耦，各自独立部署、独立伸缩。这不是历史包袱，是**刻意的分层**。

Q20 数据库 80 张表，会不会过度设计?

- **说**：81 张是按领域拆分的：账号权限、运维连接、哨兵告警、知识协同、模型网关、AI 决策闭环、团队协作七大域。拆得细是因为**审计和权限要求**——比如运行记录就拆成 `operation_run_record` / `_output` / `_evidence` 三张，用 `trace_id` 串起来，为的是**证据可独立追溯**。这不算过度设计，是合规需要的数据结构。

Q21 为什么安全判定不依赖大模型?

- **说**：这是**刻意的设计选择**。用不确定的模型去守确定的安全底线，是设计错误——模型会漂移、会幻觉、会被提示注入。所以我们的 L1 规则、L2 证据、L4 状态核验**全是确定性判定**，AI Judge 只是**可选增强**，默认还是关闭的（passthrough）。**换句话说：把模型拿掉，我们的安全机制照样工作。**
- **追问** → 那覆盖率够吗? → "规则覆盖'已知的危险'，AI 增强覆盖'没见过的语义矛盾'，两者是互补而不是替代"

Q22 前端最复杂的技术点是什么?

- **说**：两个。**一是手写 SSE**：axios 读不了流，所以我们用原生 fetch + ReadableStream + TextDecoder，按 `\n\n` 切帧解析 `thinking/token/done` 三类事件，还要支持中断。二是 **GPGPU 粒子星海**：用 ShaderMaterial 做模拟场，离屏 render 迭代后再驱动 Points，这是 GPU 上跑计算而不是单纯渲染。
- **追问** → 为什么用 Three.js? → "产品调性需要，同时证明前端工程能力；但我不夸大——它是氛围层，不是业务数据可视化"

Q23 执行一条命令的完整流程是什么? （演示题）

- **说**：五步。① **预览** `previewMultiExec` —— 先看要执行什么；② **风险评估** —— 后端按动作映射风险档（CRITICAL/HIGH/MEDIUM/LOW），命令还要过白名单和黑名单；③ **审批** —— 高危必须人工点确认，CRITICAL 直接拒绝；④ **执行** —— 沙箱/子进程，`timeout=20s`、强制 `shell=False`、输出截断 20,000 字符、禁元字符；⑤ **留痕与核验** —— 写入运行记录与审计，再由 inspector 判断是否假成功。
- **追问** → 失败了怎么办? → **诚实答**：当前上报 inspector 触发重新规划，**重试/幂等/回滚还没实现**

Q24 为什么不用 LangChain / AutoGPT 这类框架?

- **说**：三个原因。一是**可控性**——框架把规划和执行黑盒化了，而我们的核心恰恰是"每一步都可检查、可拦截"；二是**依赖**——我们要保证零密钥可跑，框架会引入大量模型和依赖；三是**课程价值**——这个项目是要讲清楚机制的，自己实现才讲得清。**代价是我们要自己实现更多东西，这个代价我们认。**

Q25 端到端闭环到底跑通了没？（高危题）

- **说**：分两条线说清楚。**产品级闭环跑通了**：后端 `DecisionLoopService` 实现了 ANALYZE → MEMORIZE → DECIDE → REVIEW → LEARNED 五阶段，有 `/api/agent/loops/{id}/{recall,decide,review}` 接口，复盘会回写 `operator_memory` 的置信度，前端有 Agent 指挥中心和执行轨迹页。**Python 侧的 planner/executor 目前是库形态**，有完整的单元测试覆盖（inspector 20 个、agent 18 个），但还没有接入 ai-service 的对话链路。**这是我们的下一步工作，我不藏着。**
- **追问** → 那你演示什么？→ "演示知识检索与对话链路（13 个单测全通过、零密钥可跑）、运维执行的预览审批执行三段式、以及决策闭环的接口"
- **别这么说**："全跑通了"——评委一看 `git grep` 就知道 planner 没调用方。

四、真实性与边界（最危险的一组，答好了反而加分）

Q26 你说有 AI，但模型是不是根本没接?

- **说**：**对，默认是 mock，这是刻意的。**配置里 `LLM_PROVIDER=mock`，Ollama 的真实调用是被注释掉的。原因很实在：**我要保证任何人拿到代码、没有任何 API Key，都能把知识检索、证据生成、可信度计算、流式输出这条链路完整跑通。**如果默认依赖真实模型，评委和老师根本跑不起来。真实模型接入是配置切换的事——`openai_compatible.py` 已经实现了完整的 `/chat/completions` 调用。
- **追问** → 那你怎么证明 AI 部分有效？→ "我证明的是**AI 之外的那一层**：检索、证据、纠察、可信度、安全拦截，这些不依赖模型且都有测试"
- **别这么说**："我们接了 GPT-4"——一眼假。

Q27 Planner 是不是就是几条 if-else?

- **说**：**是，现在是 3 条规则，最多产出 1 个步骤。**这是**确定性基线**，目的是先把链路跑通、让行为可测可复现。LLM Planner 的 prompt 模板（`PLANNER_PROMPT`）已经写好了，包括"高风险操作标记 requires_approval"和"最后必须有验证步骤"这类约束，替换只需要一个适配器。**我选择先做能测的，再做聪明的。**
- **追问** → 那算什么 Agent？→ "Agent 的核心是**规划—执行—核验—修正**的闭环结构，规划器用规则还是模型是实现选择；而且核验（四级纠察）和修正（Re-plan 建议）我们已经实现了"

Q28 你说有沙箱，用上了吗?

- **说**：**实现了，但还没接进执行器。** `sandbox/` 下有进程、文件系统、网络三个模块。当前执行走的是子进程 + `shell=False` + 超时 + 白名单这套护栏，已经能防住绝大多数风险；沙箱接线是明确的下一步。**我不把"已实现"说成"已启用"。**

Q29 审批流真的会拦住操作吗?

- **说**：**在产品链路里会拦**：`previewMultiExec` 预览之后必须走审批，审批通过才 `executeMultiExec`，前端也明确写了"审批后才会发送命令"。Python 侧的 `ApprovalManager` 目前只有数据结构、没有编排调用方——**这是两回事，我不会混着说。**另外策略引擎是硬拦截：CRITICAL 直接禁止，HIGH 且未标记需审批就拒绝执行。

Q30 Redis 你配了但没用?

- **说**：**对，依赖和配置都齐了，但代码里 RedisTemplate 引用数为 0。**这是骨架，还没落地。规划用途是会话缓存和限流。我不掩饰——**我们更愿意把"没做完"标出来，而不是让它看起来做完了。**

Q31 有没有做过实验?指标呢? （必问题）

- **说**：**没有跑过真实评测，我不会编数字。**实验文档有 17 个，其中 9 个明确标了 NOT RUN，原因写在里面：仓库没有配置真实模型、没有审核通过的数据集、没有评测器。**我能给出的真实数字是工程回归：Python 135 个用例 111 通过 24 跳过 0 失败，后端 428 个 @Test。**我们文档里自己注明这是"工程回归结果，不是模型实验结果"。
- **追问** → 那你怎么证明效果? → 接 Q32
- **别这么说**：任何形式的准确率/F1 数字。

Q32 没指标，凭什么说你的方法有效?

- **说**：三个层次。**第一，机制可验证**：假成功检测、风险拦截、版本过滤都是确定性逻辑，有单元测试直接断言（inspector 20 个用例）。**第二，链路可复现**：零密钥、13 个 knowledge 用例全通过，任何人能自己跑。**第三，边界我划得很清楚**：我只声称"链路打通、机制生效"，不声称"检索准确率提升 X%"。**我认为，在 14 条样本上硬报准确率，才是真正的不科学。**

Q33 数据集只有 14 条，还好意思叫数据集?

- **说**：**它是种子集，不是评测集**，我叫它种子集就是为了不误导。14 条覆盖 7 类运维场景，`status` 全是 `DRAFT`（未审核），`schemaVersion` 还有类型不一致的问题（4 个数字、10 个字符串）。它的唯一作用是**证明数据—检索—纠察—执行链路能打通。**要支撑性能结论，需要千级样本加人工审核，那是我们下一步明确要做的事。
- **追问** → 那你打算怎么做? → "按场景扩样到千级、双人标注一致性检验、固定 split 防泄漏（我们的 training 测试里已经有切分泄漏检查）"

Q34 前端有些页面是假数据吧?

- **说**：**有，而且我们自己标出来了。**Dashboard 首页是纯文案轮播零 API；系统知识库走本地静态条目，页面角标写着"未人工核验"；6 台 demo 服务器标注了"（模拟）"；运行记录在后端 `guideMode=true` 时会显示演示数据，前端会挂横幅声明"当前显示演示数据"；3 条 demo 成就被硬性隔离不参与统计；还有几个未注册路由的死页面（插件市场、技能市场）。**真后端的是**：个人知识库、运维执行、运行记录、模型分析、团队协作、认证与审计。

Q35 代码里有抄袭或大量 AI 生成吗?

- **说**：我们用了 AI 辅助，但**每一行都过过手**，并且有证据：全项目 grep `TODO|FIXME|HACK` **命中 0 处**；563 个测试用例覆盖核心逻辑；审计、MFA、SSH 终端、哨兵这些都是真依赖真实现（比如 SSH 用的是 `sshd-core 2.12.1`）。如果要验证，我可以当场讲任何一个模块的调用链。

Q36 你们的前端 TypeScript 是不是没开严格模式?

- **说**：**是，`strict:false`，而且类型检查只覆盖 api/domain/utils/types 四个目录，views 和 components 不在范围内。**这是历史原因——项目从 JS 起步，全量改造成本大。**这是明确的欠账**，迁移到全量 strict 在我们的技术债清单里。

Q37 trust_score 的 support_score 是不是有 bug?

- **说**：**不是 bug，是简化。**当前传的是 `bool(results)` ——有检索结果就算"支持"，这不是真的"答案—证据蕴含判定"。**真正的做法应该用 NLI 或证据蕴含模型来判定答案是否真的被证据支持。**我把它列为待改进项，并且主动标出来了。**如果老师认为这个分量当前不可靠，我完全接受——这也正是我不拿它去报指标的原因。**

五、安全与伦理

Q38 如果 AI 误操作删库怎么办?

- **说**：四层防护。① **分级**：`database.drop` 、`server.delete` 判 CRITICAL，**一律禁止**，不是"需审批"而是直接拒绝。② **命令层**：`rm` 、`dd` 、`mkfs` 、`drop` 全在黑名单，元字符 `| && ; >` 也禁。③ **执行层**：`shell=False` 不给 shell 解释、20 秒超时、输出截断。④ **事后**：审计记录写入 `operation_audit_event` ，含审批状态。另外我们还有备份与回滚的规划——**但我要诚实：回滚目前还没实现。**

Q39 谁来对 AI 的决定负责?

- **说**：**人。**我们的主张就是"AI 负责思考，人负责授权，系统负责留痕"。任何 HIGH 以上风险都必须人工点确认，审计表里记录 actor、action、target、outcome 和 approval_state，**责任链是完整可追溯的。**AI 不承担责任，它只是建议方。

Q40 会不会被用来做坏事?

- **说**：这是我们要防的。我们的机制本身就是**收窄能力**而不是放大能力——白名单只有 22 条只读类命令，黑名单覆盖 23 条破坏性命令，还有元字符和危险参数禁用。**一个只能读、不能删、不能装包、不能起容器的 Agent，攻击面是很小的。**另外内部接口有 Token 鉴权，哨兵 webhook 有 SSRF 防护（默认禁内网）。

Q41 用户数据（服务器凭证）怎么保护?

- **说**：凭证存在 `operation_credential` 表，走加密存储；SSH 连接用 `operation_known_host` 做主机校验；会话用 HttpSession + CSRF Token；管理员还有 TOTP 双因素。截图取证链路里也有 `privacyStatus` 字段做隐私状态标记。

Q42 有没有考虑提示词注入?

- **说**：**有意识但没做完。**我们的防线主要在**输出侧和执行侧**：即使模型被注入，命令也过不了白黑名单、过不了风险分级、过不了审批。这是"**不信任模型输出**"的架构。输入侧的注入检测（比如用户让 Agent 忽略规则）目前没有专门模块——**这是待补的。**

六、产品与商业

Q43 谁来付钱？怎么收？

- **说**：付钱的是**中小企业 IT 负责人**，收法是**私有化部署 + 年费订阅**。理由：运维数据敏感，他们不接受 SaaS；同时我们支持"零密钥本地跑"，部署成本极低。另有一条按次计费的路子给模型网关。

Q44 市场规模多大？

- **说**：**不给万亿级数字，只给自下而上的测算。**逻辑是：目标客户数 × 可承受客单价。我们瞄准"有专职运维但不足 3 人的企业"，这类企业在国内是百万量级；客单价按私有化部署 3–8 万/年测算。**宁可把数说小说实，也不用行业总规模去套。**

Q45 竞品是谁？你怎么活？

- **说**：三类。**通用大模型**（会说不会做、不敢授权）、**低代码 Agent 平台**（Coze/Dify 类，缺领域约束和运维能力）、**传统 AIOps**（重、贵、要专职团队）。我们活在中间：**比通用方案可信、比 AIOps 轻量。**我们的壁垒不是模型——**模型谁都能调**——而是**约束工程 + 领域知识资产 + 审计合规适配**，这些是要靠时间和场景堆的。

Q46 大厂明天抄你怎么办？

- **说**：抄功能容易，抄**约束工程的细节**难——白名单怎么定、风险怎么分级、什么样的证据算充分、假成功怎么判定，这些都是踩坑踩出来的。更重要的是**切换成本**：一旦企业的审计记录和权限体系接进来，换系统的代价很高。而且我们聚焦的是大厂看不上的中小企业长尾。

Q47 只准留一个功能，你留哪个？

- **说**：**假成功检测。**因为它是唯一一个"别人没有、且直接对应真实损失"的能力。检索大家都有，审批大家都能加，但**"AI 说做成了其实没做成"这个问题，市面上几乎没人做机器判定。**

Q48 三个月内能做到什么程度？

- **说**：三件事。① **接真实 embedding 与 LLM**，把中文检索和规划做实；② **把 Agent 内核接进对话链路**，形成完整的 Python 侧闭环；③ **建评测集**，千级样本 + 人工标注 + 固定 split，把 Recall@K、假成功检出率、风险分级准确率这几个指标真正跑出来。

七、团队与个人

Q49 你个人做了哪些？

- **说**：**按模块说，别按百分比说。**例："我负责 AI 侧：混合检索、版本过滤、可信度聚合、四级纠察的设计与实现，以及对应的单元测试。"——这样说是可验证的。**如果老师追问，我能直接翻到文件和行号。**

- **别这么说**："我做了 70%"——不可验证。

Q50 团队几个人，怎么分工?

- **说**：按层分工——后端与数据、AI 与算法、前端与交互、文档与答辩。每个人的产出都能对应到具体目录和提交记录。

Q51 遇到最大的技术困难是什么，怎么解决的?

- **说**：**挑一个真困难讲完整。**推荐讲"假成功怎么判定"：最初的困惑是"AI 说完成了，怎么知道是不是真的完成"，试过只看返回文本（会被话术骗）、只看退出码（命令成功≠问题解决），最后确定**必须引入外部实际状态做比对**，才有了 L4。这个故事的结尾要落到"**我们把不可验证的话术变成了可验证的状态比对**"。

Q52 有指导老师吗？老师给了什么帮助?

- **说**：**如实说**，并具体化（方向把关、评测方法建议、真实场景反馈）。**不要说"老师全程指导"**——那会让人怀疑你的工作量。

八、刁钻题（答好了直接加分）

Q53 给你 100 万，你怎么花?

- **说**：**别撒胡椒面。**我的分法：40 万建评测集与真实场景试点（这是当前最大短板）、30 万接真实模型与算力（embedding + 推理）、20 万做合规适配（等保、审计导出）、10 万留作安全储备。**理由一句话：我们现在最缺的不是功能，是"能证明效果的数据"。**

Q54 你这个项目最大的失败风险是什么?

- **说**：**说一个真的。**"最大的风险是**评测跟不上**——机制设计得再好，没有可信指标就无法说服用户，也无法迭代。这也是我把 100 万的最大一块投在评测上的原因。第二个风险是**中文检索效果**，在哈希嵌入下它是有上限的，必须换真 embedding。"

Q55 给自己打几分?

- **说**：**给分必给理由。**"工程完整度我给 8 分——全栈真实现、563 个测试、零密钥可复现；算法深度我给 6 分——机制设计是我最满意的部分，但多个组件还是确定性基线；评测严谨度我给 4 分——这是我承认的最大短板。**综合我给 6.5 分，扣分主要扣在没有真实指标上。**"
- **别这么说**："我给 9 分"——显得没自知之明。

Q56 如果评委说你的项目"没有算法创新"，你怎么回应?

- **说**："我理解这个质疑，因为我们的创新**不在模型结构，而在约束机制。**如果算法的评判标准是'提出了新的网络结构或损失函数'，我们确实不符合。但如果看'**为特定领域的失败模式设计可判定的算法机制**'——版本感知过滤、四级纠察、假成功状态核验、三分量可信度聚合——这些是有明确输入输出和判定逻辑的算法。**我想请老师看的是第 4 节那张算法表。**"

- **要点**：**不卑不亢，把评判标准重新定义到你的地盘上。**

Q57 你刚才那个回答是不是在回避问题?

- **说**：**直接认，然后给已知、划边界。**"您说得对，这一点我确实没有做到。我目前做到的是 X，没有做到的是 Y，Y 的实现路径是 Z，难点在 W。"
- **万能句式**：**承认 → 给已知 → 划边界 → 给路径。** 评委要的不是完美，是诚实。

Q58 你们和队友的决策层（多智能体）是什么关系?

- **说**：**按真实架构答**——"我们的 Agent 内核是按角色拆分的：planner 规划、executor 执行、inspector 核验、policies 定风险、kernel 管运行时、tools 管工具体系。当前是**流水线式的角色分工**，不是多个 Agent 互相协商。多智能体协商是我们的演进方向，但当前我们更看重**可判定性**——协商会让行为不可预测，而安全场景需要确定性。"
- **别这么说**："我们是多智能体协同系统"（当前不是协商式）。

九、收尾与通用救场

结尾（30 秒，必须有记忆点）

> 最后我想说：**这个项目的价值不在于我们让 AI 多会说话，而在于我们让它在说错、做错的时候，能被发现、被拦住、被追责。** 我们承认还有很多没做完的地方——评测、真实模型、中文分词——我们没有把它们藏起来。**但正因为我们清楚哪里没做完，我们才敢说：剩下的部分我们知道怎么做。** 谢谢各位老师。

万能救场句式（任何答不上来的题）

1. **接住**："这个问题问得很关键。"
2. **给已知**："我目前做到的是……"（说真实的部分）
3. **划边界**："……这一点我确实还没做到。"
4. **给路径**："我的解决思路是……，难点在……"
5. **反邀请**（加分）："如果老师有更成熟的做法，我非常想请教。"

上场前最后 5 分钟看这三样

1. **一页数字卡**（知识要点手册第 9 节）
2. **真相清单**（知识要点手册第 8 节）—— 8 个"没做完"，主动认
3. **三个为什么**（知识要点手册第 10 节）—— 版本感知 / 假成功 / 不依赖 LLM